C8 Copilot and AI in Fabric

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track C: Microsoft Fabric 2

C8 Copilot and AI in Fabric 3

C8-L1 Copilot in Fabric, where it runs and what it costs 3

C8-L2 Data agents, questions in plain language 7

C8-L3 Verify what Copilot writes 12

Track C best-practice checklist 17

Appendix: solutions and answer keys 18

Track C: Microsoft Fabric

Unify data, analytics and Copilot in Fabric, and verify what Copilot produces.

C8 Copilot and AI in Fabric

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Copilot across workloads, data agents, limits and verification.

C8-L1 Copilot in Fabric, where it runs and what it costs

Concept

Copilot in Fabric is an AI assistant built into each workload. It writes notebook code, pipelines, SQL, KQL and DAX, and summarises reports. Before you rely on it, check three things.

1. Is it switched on for you? Copilot needs a paid Fabric capacity of **F2 or higher**, or Power BI Premium **P1 or higher**, in a supported region. Fabric administrators control it through tenant settings, scoped to security groups. If your capacity's region lacks Azure OpenAI, an admin must also allow data to be processed outside that region. Copilot stops working when a Fabric trial expires.

2. What is its status? Release status differs by workload (Microsoft Learn, checked September 2026):

Experience	Status
Copilot in notebooks (Data Engineering, Data Science)	Preview
Data agents	GA
Copilot in Data Factory (Dataflow Gen2, pipelines)	GA
Copilot for SQL queries in a Warehouse	Preview
Copilot for semantic models and reports in Power BI	GA

3. What does it cost? Copilot requests consume capacity units, metered per 1,000 tokens. About 1,000 tokens equal 750 words. Input costs 100 CU seconds per 1,000 tokens, cached input 10 and output 400. A request with 2,000 input tokens and 500 output tokens costs (2,000 × 100 + 500 × 400) / 1,000 = 400 CU seconds. Copilot runs as a **background** operation, so Fabric smooths its cost over 24 hours.

On data handling, Microsoft states your data is not used to train models. It also states that Copilot output is sometimes inaccurate, and that people able to judge its accuracy should review it. Lesson C8-L3 shows you how.

Why it matters

An analytics team put Copilot-generated SQL into a scheduled report during the Warehouse Copilot preview. Nobody recorded which features were preview. When a reviewer asked whether the process relied on anything outside GA, the team spent a week tracing it. Check status first, record it, and keep production logic reviewed and owned by a person.

Worked example: weak code

```
1 Plan: use Copilot to write all transformation SQL
1 Capacity: trial
2 Review: none, Copilot output goes straight to the scheduled job
3 Cost estimate: Copilot is included
```

1. **Line 2.** Copilot stops working when the trial expires.
2. **Line 3.** Microsoft states output is sometimes inaccurate. Someone qualified must review it.
3. **Line 4.** Every request consumes CUs from your capacity.

Worked example: strong code

```
1 Plan: Copilot drafts SQL, an engineer reviews, tests and owns it
2 Capacity: F8, Copilot tenant setting scoped to the data-engineering group
2 Status check: Warehouse Copilot = preview (Learn, September 2026), noted
    in the design record
3 Cost estimate: about 400 CU seconds per request, smoothed over 24 hours
```

1. **Line 1.** Copilot drafts. A person decides.
2. **Line 3.** Preview features change. Record the status and the date.
3. **Line 4.** Estimated from the published token rates.

Confirm Copilot is enabled on a paid capacity, record the release status of each experience, and budget its CU use.

Recap card

- **Copilot in Fabric capacity requirement** Paid F2+ or P1+ in a supported region, with tenant settings enabled. Not after a trial expires.
- **Copilot release status** Per experience. Notebooks and Warehouse SQL preview. Data agents, Data Factory and Power BI GA (September 2026).
- **Copilot token rates per 1,000 tokens** Input 100, cached input 10, output 400 CU seconds. Background job, smoothed over 24 hours.
- **Copilot output** Sometimes inaccurate. A person able to judge it reviews it before use.

Practice

Exercise 1. Multiple choice (level 1)

Which capacity runs Copilot in Fabric?

- a) Any capacity, including an expired trial
- b) A paid F2 or higher, or P1 or higher, in a supported region
- c) Only F64 or higher
- d) No capacity: Copilot runs on the user's licence

Why?

- r1) Microsoft Learn lists a paid F2+ or P1+ capacity in a supported region, with tenant settings enabled.
- r2) Copilot is part of every Power BI Pro licence.

Exercise 2. Multiple choice (level 2)

Your design uses Copilot in notebooks and Copilot in Data Factory pipelines. What do you record about their status?

- a) Both GA
- b) Notebooks preview, Data Factory GA, with the date you checked
- c) Both preview
- d) Nothing: Copilot has one status across Fabric

Why?

- r1) Microsoft Learn's release-status page lists each Copilot experience separately.
- r2) Everything in Data Factory is GA.

Exercise 3. Fill the gap (level 2)

Complete the Copilot cost for 1,000 input tokens and 1,000 output tokens, with no cache.

```
input:  1,000 tokens x 100 / 1,000 = 100 CU seconds
output: 1,000 tokens x _____(1) / 1,000 = _____(2) CU seconds
total: 500 CU seconds
```

Exercise 4. Write the function (level 3)

Write `copilot_cu_seconds(input_tokens: int, output_tokens: int, cached_input_tokens: int = 0) -> float`. `cached_input_tokens` is the part of `input_tokens` served from the prompt cache. Rates per 1,000 tokens: input

100, cached input 10, output 400 CU seconds. Raise `ValueError` if any count is negative or if cached tokens exceed input tokens.

Your code must pass these checks:

- 2,000 input and 500 output tokens cost 400 CU seconds. Output is 400 per 1,000.
- Cached input costs 10 per 1,000, and those tokens are not charged at the full input rate as well.
- Cached tokens are part of input tokens. More cached than input is invalid.
- Negative token counts are invalid.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
INPUT_RATE = ...

# Module constant.
CACHED_INPUT_RATE = ...

# Module constant.
OUTPUT_RATE = ...

# Module constant.
TOKENS_PER_UNIT = ...

# CU seconds for one Copilot request at the published token rates.
def copilot_cu_seconds(
    input_tokens: int,
    output_tokens: int,
    cached_input_tokens: int = 0,
) -> float: ...
```

Starter code:

```python
def copilot_cu_seconds(input_tokens, output_tokens,
   cached_input_tokens=0):
     return (input_tokens + output_tokens) * 100 / 1000
```

Exercise 5. Architecture design (level 4)

Your organisation wants Copilot for its data-engineering team only, on an F16 capacity, with an audit trail of which Copilot features the pipelines depend on.

Options: Enabled for the data-engineering security group, Enabled for the whole organisation.

1. Tenant setting scope
2. Design record

3. Copilot output in production

Sources

- Copilot requires a paid F2+ or P1+ capacity in a supported region; tenant settings scoped to security groups; cross-geo processing settings; stops after trial expiry. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-enable-fabric (updated 2026-08-10); troubleshoot-copilot-unavailable-after-trial (updated 2026-07-10))
- Release status: notebooks preview, Data Factory GA, Warehouse SQL Copilot preview, Power BI semantic models and reports GA. Data agent GA per its concept page; the release-status page lists it in a Data Science row marked preview alongside AI functions. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-ai-feature-state (updated 2026-06-19); data-science/concept-data-agent (updated 2026-05-11))
- Copilot consumption: 100 CU seconds per 1,000 input tokens, 10 cached input, 400 output; about 1,000 tokens = 750 words; background operation smoothed over 24 hours; worked example 400 CU seconds. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-fabric-consumption (updated 2026-05-22))
- Data not used to train models; output is sometimes inaccurate and should be reviewed by people able to evaluate it. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-privacy-security (updated 2025-06-25))

C8-L2 Data agents, questions in plain language

Concept

A **Fabric data agent** (GA, Microsoft Learn, checked September 2026) answers questions in plain language over your data. It picks a data source, writes a query and runs it:

- **SQL** for lakehouses and warehouses.
- **DAX** for Power BI semantic models.
- **KQL** for KQL databases in an eventhouse.

One agent takes up to **five data sources**. Users do not write the query, but the query is still what decides the answer.

Three rules keep it safe:

- **Read-only.** The agent generates only read queries. It never creates, updates or deletes data, and it does not run notebooks or other write workflows.
- **The user's permissions.** The agent runs with the credentials of the person asking, so each user only reaches data they already have access to.
- **No unstructured files.** PDFs, Word documents and text files are not supported sources.

The agent is only as good as the context you give it. Microsoft's guidance:

- **Select only the tables and columns** the agent needs. Every extra table is another chance to pick the wrong one.
- **Use clear names** such as `order_submission_date`, not `date1`.
- **Write instructions** that explain business terms and which source answers which kind of question.
- **Add example queries**: question and query pairs. Each example is validated against the schema, and invalid ones are not used. Lakehouses, warehouses and KQL databases support examples. Semantic models do not.

Then **evaluate** the agent. Build a ground-truth set of questions with known answers, run them through the agent, and measure how many it gets right. The `fabric-data-agent-sdk` package provides an evaluation API for this. Rerun the set after every change to instructions, examples or schema.

Every query costs tokens. Instructions, example queries and conversation history all add to the input and so to CU use.

Why it matters

A sales team published a data agent over a lakehouse with 60 tables, including three with a column called `revenue`: gross, net and forecast. "What was revenue last quarter?" returned the forecast. Nobody noticed for a month. Selecting only the net-revenue table, plus an instruction defining revenue as net of returns, fixed it. A 20-question ground-truth test would have caught it on day one.

Worked example: weak code

```
1 Sources: sales_lakehouse (all 60 tables)
2 Instructions: (none)
3 Example queries: (none)
```

```
❷ Testing: tried three questions in the chat, looked fine
```

1. **Line 1.** Three revenue columns. The agent picks one.
2. **Line 4.** Looking fine is not a measured accuracy.

Worked example: strong code

```
❶ Sources: sales_lakehouse (fact_sales, dim_customer, dim_date only)
❷ Instructions: "Revenue" means net_revenue_gbp, after returns. Quarters are
    calendar quarters.
  3 Example queries: 12 question and SQL pairs, all passing schema validation
❸ Testing: 40 ground-truth questions, rerun after every change, accuracy
    recorded
```

1. **Line 1.** Fewer tables, fewer wrong choices.
2. **Line 2.** Business terms defined once, for every question.
3. **Line 4.** A number you track, not a feeling.

Narrow the schema, define your terms, add validated examples, and measure accuracy against known answers.

Recap card

- **Data agent query languages** SQL for lakehouse and warehouse, DAX for semantic models, KQL for KQL databases. Up to five sources.
- **Data agent security** Read-only queries only. Runs with the asking user's permissions.
- **Example queries** Question and query pairs, validated against the schema. Not supported for semantic models.
- **Measure a data agent** Ground-truth questions with known answers, rerun after every change, accuracy recorded.

Practice

Exercise 1. Multiple choice (level 1)

A user asks a data agent to "delete the test orders". What happens?

- a) The agent writes and runs a DELETE statement
- b) The agent generates only read queries, so it does not delete data
- c) The agent deletes them if the user is a workspace Admin
- d) The agent asks the data owner for approval

Why?

- r1) Microsoft Learn states the data agent generates only read queries and keeps read-only connections.

- r2) The agent has no permissions of its own.

Exercise 2. Multiple choice (level 2)

Ana has access to the UK sales table but not the EU one. Both are sources of the same data agent. Ana asks for total EU sales. What does the agent do?

- a) Returns EU sales, because the agent's owner has access
- b) Answers only from data Ana has permission to read
- c) Returns EU sales with the numbers masked
- d) Returns a random estimate

Why?

- r1) The agent uses the requesting user's credentials and permissions.
- r2) The agent filters rows by country automatically.

Exercise 3. Put in order (level 2)

Order the steps to build a data agent over a sales lakehouse.

- Select only the tables and columns the questions need
- Add example question and SQL pairs
- Publish and share the agent
- Run the ground-truth question set and record accuracy
- Write instructions that define business terms

Exercise 4. Write the function (level 3)

Write `evaluate(expected: dict[str, float], actual: dict[str, float | None], rel_tol: float = 0.001) -> tuple[float, list[str]]`. `expected` maps question ids to ground-truth answers. `actual` maps them to the agent's answers, where `None` means the agent gave no number. Return the share of questions answered correctly (within `rel_tol`, and exactly for an expected 0.0) and a sorted list of the failed ids. A question missing from `actual` fails. Raise `ValueError` if `expected` is empty.

What you need to know.

`math.isclose(a, b, rel_tol=0.001)` is `True` when `a` and `b` differ by at most 0.1% of the larger of the two. Use it for floats, where `==` fails on rounding noise. With its default absolute tolerance of 0, a comparison with 0.0 is true only for exactly 0.0.

```python
import math

```

```
math.isclose(100.0, 100.05, rel_tol=0.001)   # True
math.isclose(0.0, 1e-12)                     # False
```

Your code must pass these checks:

- q2 is 10% off, so it fails. One of two correct gives 0.5.
- An answer of 0.0 is a real answer. Test for None with `is None`, not truthiness.
- None and missing answers fail, and accuracy divides by the number of expected questions.
- An empty ground-truth set has no accuracy. Raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Accuracy of a data agent against a ground-truth set.
def evaluate(
    expected: dict[str, float],
    actual: dict[str, float | None],
    rel_tol: float = 0.001,
) -> tuple[float, list[str]]: ...
```

Starter code:

```python
def evaluate(expected, actual, rel_tol=0.001):
    failed = [q for q in expected if not actual.get(q)]
    return 1 - len(failed) / len(actual), failed
```

Exercise 5. Architecture design (level 4)

Configure a data agent for finance analysts. Sources: a finance warehouse and a Power BI semantic model of the same numbers. Contracts in PDF live in a lakehouse Files folder.

Options: Leave them out: unstructured files are not supported, Add the Files folder as a source.

1. The PDF contracts
2. Where to add example queries
3. Instruction to add

Sources

- Data agent is GA per its concept page; supports up to five sources (lakehouse, warehouse, semantic model, KQL database, and others); generates SQL, DAX, KQL; read-only; uses the requesting user's credentials; no unstructured data such as PDF, docx, txt. (verified: https://learn.microsoft.com/en-us/fabric/data-science/concept-data-agent (updated 2026-05-11); copilot-ai-feature-state (updated 2026-06-19))

- Select only needed tables and columns; use descriptive names; instructions and descriptions supply business meaning. (verified: https://learn.microsoft.com/en-us/fabric/data-science/data-agent-configuration-best-practices (updated 2026-08-27))
- Example queries are validated against the schema and supported for lakehouse, warehouse and KQL databases, not semantic models or ontology. (verified: https://learn.microsoft.com/en-us/fabric/data-science/data-agent-example-queries (updated 2025-09-11))
- fabric-data-agent-sdk provides evaluation against ground-truth question sets. (verified: https://learn.microsoft.com/en-us/fabric/data-science/evaluate-data-agent (updated 2026-09-01))
- Instructions, example queries and conversation history add to token usage and CU consumption. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/data-agent-consumption (updated 2025-03-20))

C8-L3 Verify what Copilot writes

Concept

Microsoft's own guidance for Copilot in notebooks (preview, Microsoft Learn, checked September 2026) says: always review AI-generated code before you apply it. Output might be inaccurate, incomplete, or based on outdated library syntax. The chat pane shows changes as an approval diff. Approving the diff is your review, so make it a real one.

Code that runs without error proves only that the names exist and the syntax parses. Use four checks, in this order.

1. **Read every line.** Check each table, column and function against the schema and the docs. Check the grain: one row per what? Check joins: inner joins drop unmatched rows.
2. **Run it on known data.** Build five to ten rows where you have worked out the answer by hand. Include the awkward cases: a null, a zero denominator, a duplicate key, an empty group.
3. **Compare with a second method.** Compute the same number another way: a plain `count()` against the generated aggregate, SQL against PySpark, the new query against last month's report.
4. **Keep the checks.** Turn steps 2 and 3 into tests that run with the pipeline, so the next edit, by you or by Copilot, is checked too.

Errors to look for first, because they run cleanly and give plausible numbers:

- **Average of averages.** `AVG(avg_basket)` over stores weights a small store the same as a large one. Divide total sales by total baskets instead.
- **Wrong join type.** An inner join to a dimension drops facts with no match.
- `COUNT(*)` **for distinct things.** Customers with two orders count twice.
- **Unguarded division.** A zero denominator fails in some engines and returns null or infinity in others.
- **Invented names.** A column or function that does not exist. This one fails loudly.

Why it matters

An analyst asked Copilot for average basket value by region. The query ran and the numbers looked sensible. It averaged each store's average, so a pop-up store with four baskets counted as much as a flagship with 40,000. The regional figure was 11% too high. A five-row test with two stores of different sizes shows the difference at once.

Worked example: weak code

```
1 # Copilot suggestion, applied after it ran without error
2 region_basket = (
3     store_summary.join(dim_region, "store_id")
4     .groupBy("region")
2     .agg(F.avg("avg_basket").alias("avg_basket"))
6 )
```

1. **Line 1.** Running is not a check.
2. **Line 5.** Average of averages: every store weighs the same.

Worked example: strong code

```
1  from pyspark.sql import functions as F
2
3  region_basket = (
4      store_summary.join(dim_region, "store_id", "left")
5      .groupBy("region")
1      .agg(F.try_divide(F.sum("total_sales"),
       F.sum("basket_count")).alias("avg_basket"))
7  )
8
9  # Known data: store A 4 baskets, 40.00 total. Store B 400 baskets,
     2,000.00 total.
2  # Hand answer: 2,040.00 / 404 = 5.0495...  Average of averages would give
     7.50.
```

```
11 assert abs(result_for("North") - 2040.0 / 404) < 1e-9
```

1. **Line 6.** Total over total: each basket counts once. try_divide returns null for a zero denominator, where plain division fails under ANSI mode.
2. **Line 10.** Known data picked so the wrong method gives a visibly different answer.

Read it, run it on data with a hand-worked answer, compare it with a second method, and keep those checks as tests.

Recap card

- **A Copilot query ran without error. What does that prove?** Syntax parses and names exist. Nothing about correctness.
- **Four checks for Copilot output** Read every line. Run on known data. Compare with a second method. Keep the checks as tests.
- **Average of averages** Weights small groups like large ones. Divide total by total instead.
- **Known data for verification** Five to ten hand-built rows with a null, a zero denominator, a duplicate key and an empty group.

Practice

Exercise 1. Multiple choice (level 1)

A Copilot-generated query runs without an error. What has that shown?

- a) The results are correct
- b) The syntax parses and the referenced names exist
- c) The query is efficient
- d) The grain matches the requirement

Why?

- r1) An engine checks syntax and names before it runs a query. It does not know what answer you wanted.
- r2) Copilot tests its own queries before it shows them.

Exercise 2. Spot the bug (level 2)

Copilot wrote this T-SQL for average basket value by region. `store_summary` has one row per store with `total_sales`, `basket_count` and `avg_basket`. Tap the line that gives the wrong number, then fix it. Guard the division.

```
1 SELECT r.region,
2        AVG(s.avg_basket) AS avg_basket
3 FROM dbo.store_summary AS s
```

```
4 LEFT JOIN dbo.dim_region AS r ON s.store_id = r.store_id
5 GROUP BY r.region;
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Put in order (level 2)

Order the checks for a Copilot-generated transformation.

- Run it on a small dataset with a hand-worked answer
- Compare the result with a second method on real data
- Keep the known-data and comparison checks as tests
- Read every line: names, grain, join types

Exercise 4. Write the function (level 3)

Write `mismatches(candidate: Callable[[list[float]], float | None], reference: Callable[[list[float]], float | None], cases: list[list[float]]) -> list[int]`. Run both functions on each case. Return the indexes of cases where the candidate raises an exception, or where the results differ. Two `None` results match. Floats match within `math.isclose` defaults. Import `Callable` from `collections.abc`.

What you need to know.

`Callable` from `collections.abc` types a parameter that holds a function. `Callable[[list[float]], float | None]` takes a list of floats and returns a float or `None`. Call the parameter like any function.

`try: ... except Exception:` catches any error the call raises, so one failing case does not stop the loop:

```python
from collections.abc import Callable

def safe(fn: Callable[[int], float], x: int) -> float | None:
    try:
        return fn(x)
    except Exception:
        return None
```

Your code must pass these checks:

- Report the index where the results differ.
- A candidate that raises on a case is a mismatch, not a crash of the harness.
- 0.1 + 0.2 and 0.3 differ in the last bit. Compare floats with math.isclose.
- None and 0.0 are different results.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Indexes of cases where a candidate function disagrees with a reference.
def mismatches(
    candidate: Callable[[list[float]], float | None],
    reference: Callable[[list[float]], float | None],
    cases: list[list[float]],
) -> list[int]: ...
```

Starter code:

```python
def mismatches(candidate, reference, cases):
    return [i for i, case in enumerate(cases) if candidate(case) !=
    reference(case)]
```

Exercise 5. Architecture design (level 4)

Copilot wrote a PySpark step that deduplicates survey respondents and computes weighted reach per wave. Plan its verification before it goes into the silver pipeline.

Options: 8 rows: a duplicate respondent, a null weight, a wave with zero total weight, A 1% random sample of production.

1. Known dataset
2. Second method
3. After it passes

Sources

- Copilot for Data Engineering and Data Science is preview; always review AI-generated code; output might be inaccurate, incomplete or based on outdated library syntax; chat pane uses an approval diff. (verified: https://learn.microsoft.com/en-us/fabric/data-engineering/copilot-notebooks-overview (updated 2026-04-01))
- With ANSI mode on, Spark division by zero raises DIVIDE_BY_ZERO; F.try_divide returns null instead. (verified: Local run on PySpark 4.x (practice venv), September 2026; error message names try_divide.)

Track C best-practice checklist

☐ Check the status (GA or preview) of every Fabric feature before you depend on it.

☐ One copy of each dataset. Use shortcuts or mirroring before you copy.

☐ Lakehouse for Spark and files. Warehouse for T-SQL writes and multi-table transactions.

☐ The SQL analytics endpoint is read-only for table data. Write through Spark or a Warehouse.

☐ Semantic models use star schemas. DAX measures state their filter context.

☐ Direct Lake models are checked for fallback to DirectQuery.

☐ Every Copilot output is read, run on known data and compared against a second method.

☐ Workspaces map to domains. Access uses roles and OneLake security, not item sharing.

☐ Items live in Git. Changes move through deployment pipelines.

☐ Capacity use is monitored, and heavy jobs are scheduled off peak.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

C8-L1 Copilot in Fabric, where it runs and what it costs

C8-L1 exercise 1

Hints. (1) Copilot consumes CUs. Where do CUs come from? (2) Paid, not trial. (3) F2 or P1 and up.

Answer. b) A paid F2 or higher, or P1 or higher, in a supported region Because: r1) Microsoft Learn lists a paid F2+ or P1+ capacity in a supported region, with tenant settings enabled.

Paid F2+ or P1+, a supported region, and tenant settings switched on.

Common wrong answers.

- `a`: Copilot stops working when a trial expires.
- `c`: F64 matters for free-licence viewers (lesson C1-L1). Copilot starts at F2.
- `d`: Copilot consumes capacity units from the workspace's capacity.
- `b|r2`: Right answer, wrong reason. Copilot depends on the capacity and tenant settings, not a Pro licence.

C8-L1 exercise 2

Hints. (1) Is Copilot one feature or many? (2) Check the release-status page per workload. (3) Record the date: status changes.

Answer. b) Notebooks preview, Data Factory GA, with the date you checked Because: r1) Microsoft Learn's release-status page lists each Copilot experience separately.

Status is per experience. Record each one with the date you checked.

Common wrong answers.

- `a`: Copilot in notebooks is preview (Learn, September 2026).
- `c`: Copilot in Data Factory is GA (Learn, September 2026).
- `d`: Status differs by workload and changes over time. Record each one with a date.
- `b|r2`: Right answer, wrong reason. Status is per feature. Some Data Factory features are preview.

C8-L1 exercise 3

Hints. (1) Output and input rates differ. (2) Output is four times the input rate. (3) 400 and 400.

Answer.

```
input:  1,000 tokens x 100 / 1,000 = 100 CU seconds
output: 1,000 tokens x 400 / 1,000 = 400 CU seconds
total: 500 CU seconds
```

Output costs 400 CU seconds per 1,000 tokens. Long answers cost more than long questions.

Common wrong answers.

- g1 = `100`: Output tokens cost more than input: 400 CU seconds per 1,000.

C8-L1 exercise 4

Hints. (1) Three rates: 100, 10 and 400 per 1,000 tokens. (2) Uncached input = input_tokens - cached_input_tokens. (3) Validate before you compute.

Answer.

```python
INPUT_RATE = 100
CACHED_INPUT_RATE = 10
OUTPUT_RATE = 400
TOKENS_PER_UNIT = 1000


def copilot_cu_seconds(
    input_tokens: int, output_tokens: int, cached_input_tokens: int = 0
) -> float:
    """CU seconds for one Copilot request at the published token rates.

    Args:
        input_tokens: All input tokens, cached or not.
        output_tokens: Output tokens.
        cached_input_tokens: Input tokens served from the prompt cache.

    Returns:
        Capacity unit seconds consumed.

    Raises:
        ValueError: If a count is negative or cached exceeds input.
    """
    if min(input_tokens, output_tokens, cached_input_tokens) < 0:
        raise ValueError("token counts must not be negative")
    if cached_input_tokens > input_tokens:
        raise ValueError("cached tokens cannot exceed input tokens")
    uncached = input_tokens - cached_input_tokens
    total = (
```

```
        uncached * INPUT_RATE
        + cached_input_tokens * CACHED_INPUT_RATE
        + output_tokens * OUTPUT_RATE
    )
    return total / TOKENS_PER_UNIT
```

Charge each token once, at its own rate. Put the rates in named constants so you update one place when prices change.

C8-L1 exercise 5

Hints. (1) Tenant settings take security groups. (2) Is Copilot one feature? (3) Who owns production logic?

Answer.

- Scope the tenant setting to the security group that needs Copilot.
- Record each experience with its status and date, since status differs by workload.
- Treat Copilot output as a draft: a person reviews, tests and owns it.

Scoped access, a dated status record, and human ownership of every output.

Common wrong answers.

- scope = `everyone`: Scope tenant settings to the groups that need them. Every request draws on the F16.
- record = `copilot_yes`: Status is per experience. 'Yes' tells an auditor nothing.
- prod = `direct`: Running without error is not correctness. Lesson C8-L3 shows how to check.

C8-L2 Data agents, questions in plain language

C8-L2 exercise 1

Hints. (1) Which query types does the agent generate? (2) Only read queries. (3) Read-only holds for Admins too.

Answer. b) The agent generates only read queries, so it does not delete data Because: r1) Microsoft Learn states the data agent generates only read queries and keeps read-only connections.

Data agents are read-only by design.

Common wrong answers.

- `a`: The agent never generates create, update or delete queries.
- `c`: Read-only applies to every user, whatever their role.

- `d`: There is no approval flow. The agent does not write.
- `b|r2`: Right answer, wrong reason. The agent uses the user's permissions. It is read-only by design, whatever those permissions allow.

C8-L2 exercise 2

Hints. (1) Whose credentials does the agent use? (2) The person asking. (3) Ana cannot read EU sales.

Answer. b) Answers only from data Ana has permission to read Because: r1) The agent uses the requesting user's credentials and permissions.

The agent never widens access. Grant permissions on the data, not on the agent.

Common wrong answers.

- `a`: The agent runs as the person asking, not as its owner.
- `c`: The agent applies the user's permissions. It does not add its own masking.
- `d`: The agent queries data. It does not invent estimates for data it cannot read.
- `b|r2`: Right answer, wrong reason. There is no automatic country filter. Ana's own permissions decide what she reaches.

C8-L2 exercise 3

Hints. (1) What do examples get validated against? (2) Context before testing. (3) Test before publishing.

Answer.

1. Select only the tables and columns the questions need
2. Write instructions that define business terms
3. Add example question and SQL pairs
4. Run the ground-truth question set and record accuracy
5. Publish and share the agent

Schema, instructions, examples, evaluation, then publish.

Common wrong answers.

- Measure accuracy before users rely on the agent.
- Example queries are validated against the selected schema. Select the tables first.

C8-L2 exercise 4

Hints. (1) Use `answer is None` to detect a missing number. (2) `math.isclose` handles the tolerance. (3) Divide by `len(expected)`.

Answer.

```python
import math


def evaluate(
    expected: dict[str, float], actual: dict[str, float | None], rel_tol: float = 0.001
) -> tuple[float, list[str]]:
    """Accuracy of a data agent against a ground-truth set.

    Args:
        expected: Question id to ground-truth answer.
        actual: Question id to the agent's answer, None if it gave no number.
        rel_tol: Relative tolerance for a match.

    Returns:
        Share of expected questions answered correctly, and sorted failed ids.

    Raises:
        ValueError: If expected is empty.
    """
    if not expected:
        raise ValueError("ground-truth set is empty")
    failed = []
    for question, truth in expected.items():
        answer = actual.get(question)
        if answer is None or not math.isclose(answer, truth, rel_tol=rel_tol):
            failed.append(question)
    return (len(expected) - len(failed)) / len(expected), sorted(failed)
```

The denominator is the ground-truth set. A zero is an answer. Both mistakes inflate or deflate accuracy without an error.

C8-L2 exercise 5

Hints. (1) Which file types are supported? (2) Which source types accept example queries? (3) Two sources, same numbers: how does the agent choose?

Answer.

- Data agents do not support PDF, Word or text files.

- Example queries work on lakehouse, warehouse and KQL sources, not semantic models.
- Two sources hold the same numbers, so tell the agent which to use for which question.

Supported sources only, examples where they work, and routing instructions where sources overlap.

Common wrong answers.

- pdfs = `include`: Unstructured files such as PDFs are not supported data agent sources.
- examples = `model`: Semantic model sources do not support example queries.
- routing = `no_instr`: With two sources of the same numbers, the agent needs an instruction to route questions.

C8-L3 Verify what Copilot writes

C8-L3 exercise 1

Hints. (1) What does the engine check before running? (2) Does it know what answer you expected? (3) Syntax and names only.

Answer. b) The syntax parses and the referenced names exist Because: r1) An engine checks syntax and names before it runs a query. It does not know what answer you wanted.

A clean run is the start of verification, not the end.

Common wrong answers.

- `a`: An average of averages runs cleanly and returns a wrong number.
- `c`: A query that scans a whole table runs without error too.
- `d`: The engine does not know the grain you intended.
- `b|r2`: Right answer, wrong reason. Microsoft says to review all generated code. The engine did the checking, not Copilot.

C8-L3 exercise 2

Hints. (1) Does every store weigh the same in the result? (2) Should it? (3) Total sales over total baskets.

Answer. The bug is on line 2.

Fixed version:

```
1 SELECT r.region,
```

```sql
       SUM(s.total_sales) / NULLIF(SUM(s.basket_count), 0) AS avg_basket
FROM dbo.store_summary AS s
LEFT JOIN dbo.dim_region AS r ON s.store_id = r.store_id
GROUP BY r.region;
```

Recompute ratios from their totals at the level you report.

Common wrong answers.

- Line 4: A left join keeps stores without a region. That line is fine.
- Line 5: Grouping by region matches the requirement.

C8-L3 exercise 3

Hints. (1) Cheapest check first. (2) Which check gives you a known right answer? (3) Keep what you built.

Answer.

1. Read every line: names, grain, join types
2. Run it on a small dataset with a hand-worked answer
3. Compare the result with a second method on real data
4. Keep the known-data and comparison checks as tests

Read, known data, second method, then keep the checks.

Common wrong answers.

- Read first. A wrong join or invented column is cheaper to catch on paper.
- Known data first: it tells you which answer is right when the two methods disagree.

C8-L3 exercise 4

Hints. (1) Wrap only the candidate call in try/except. (2) Handle None before math.isclose, which rejects None. (3) `expected is not got` is True when one is None and the other is not.

Answer.

```python
import math
from collections.abc import Callable


def mismatches(
    candidate: Callable[[list[float]], float | None],
    reference: Callable[[list[float]], float | None],
    cases: list[list[float]],
) -> list[int]:
    """Indexes of cases where a candidate function disagrees with a reference.

```

```
    Args:
        candidate: Function under test, such as Copilot-generated code.
        reference: Trusted implementation.
        cases: Inputs to run through both.

    Returns:
        Case indexes where the candidate raised or returned a different
    result.
    """
    failed: list[int] = []
    for index, case in enumerate(cases):
        expected = reference(case)
        try:
            got = candidate(case)
        except Exception:
            failed.append(index)
            continue
        if expected is None or got is None:
            if expected is not got:
                failed.append(index)
        elif not math.isclose(got, expected):
            failed.append(index)
    return failed
```

A comparison harness turns "looks right" into a list of failing cases. Keep it with the pipeline tests.

C8-L3 exercise 5

Hints. (1) Which dataset has an answer you know? (2) Is the second method independent? (3) What checks the next change?

Answer.

- Hand-built rows cover the cases that break code quietly. A random sample rarely contains them.
- A second, independent method on the same rows catches logic errors.
- A committed test checks every later change, including the next Copilot edit.

Known awkward data, an independent second method, and a committed test.

Common wrong answers.

- data = `sample`: You have no hand-worked answer for a random sample, and it seldom contains the awkward cases.
- second = `rerun`: The same code gives the same wrong answer twice. The second method must be independent.

- keep = `note`: A note does not run. A test fails when a later change breaks the logic.